

TEMA 4. FUNCIONES

1. Introducción a las funciones en Python

En programación, una **función** es un bloque de código reutilizable que realiza una tarea específica y que se ejecuta únicamente cuando se invoca.

Las funciones permiten dividir un programa en partes más pequeñas y organizadas, lo que mejora la claridad, la reutilización del código y la facilidad de mantenimiento.

En lugar de repetir el mismo conjunto de instrucciones varias veces, podemos agruparlas dentro de una función y llamarla cada vez que la necesitemos.

¿Por qué son importantes las funciones?

El uso de funciones aporta principalmente:

-  **Modularidad** → el programa se divide en bloques independientes.
 -  **Reutilización** → se escribe el código una vez y se usa múltiples veces.
 -  **Mejor organización** → cada función tiene una responsabilidad concreta.
 -  **Facilidad de depuración** → se pueden probar funciones por separado.
 -  **Escalabilidad** → es más fácil ampliar el programa sin romper lo anterior.
-

Idea clave

Una función convierte un problema grande en pequeñas tareas manejables.

En lugar de tener un programa largo y difícil de entender, las funciones permiten estructurarlo en bloques claros y bien definidos.

Primer ejemplo conceptual

Sin funciones (código repetido):

```
nota1 = 6
```

```
nota2 = 8
nota3 = 7
promedio = (nota1 + nota2 + nota3) / 3
print(promedio)
```

Si necesitamos hacer esto muchas veces, el código se vuelve repetitivo.

Con función:

```
def calcular_promedio(n1, n2, n3):
    return (n1 + n2 + n3) / 3
```

Ahora podemos usarla tantas veces como queramos:

```
print(calcular_promedio(6, 8, 7))
print(calcular_promedio(4, 5, 6))
```

Conceptos que veremos en el tema

A lo largo del tema aprenderemos:

- Cómo definir funciones con `def`
 - Cómo usar parámetros
 - Qué es `return`
 - Diferencia entre imprimir y devolver valores
 - Tipos de argumentos (posicionales, nombrados, por defecto)
 - Alcance de variables (local y global)
 - Buenas prácticas y modularización
-

2. Sintaxis básica con `def`

◆ Estructura general

```
def nombre_funcion(parametros):  
    instrucciones  
    return valor # opcional
```

2.1. Elementos importantes

1 def

Palabra clave obligatoria para definir una función.

```
def saludo():
```

✗ Error común: escribir `Def` o `DEF`.

2 Nombre de la función

Debe ser descriptivo y seguir la convención `snake_case`.

```
def calcular_area():
```

3 Paréntesis

Siempre obligatorios, aunque no haya parámetros.

```
def saludar():
```

✗ Error:

```
def saludar:
```

4 Dos puntos :

Indican el inicio del bloque.

✗ Olvidarlos provoca `SyntaxError`.

5 Indentación

El cuerpo de la función debe ir indentado (4 espacios).

```
def saludar():  
    print("Hola")
```

Python usa la indentación para saber qué pertenece a la función.

6 return (opcional, pero clave)

Permite devolver un valor al punto donde fue llamada la función.

Si no hay `return`, la función devuelve automáticamente `None`.

◆ Primer ejemplo básico (sin parámetros)

```
def saludar():  
    print("Hola")
```

```
saludar()
```

Aquí la función:

- No recibe datos.
- Solo ejecuta una acción.
- No devuelve nada.

◆ Segundo ejemplo (con parámetros)

```
def saludar(nombre):  
    print("Hola", nombre)
```

```
saludar("Ana")
```

Aquí:

- `nombre` es un parámetro.
- `"Ana"` es el argumento.

◆ Tercer ejemplo (con `return`)

```
def suma(a, b):  
    return a + b
```

```
resultado = suma(3, 5)  
print(resultado)
```

Aquí la función:

- Calcula.
- Devuelve el resultado.
- Permite reutilizar ese valor.

Diferencia clave

Función que imprime:

```
def suma(a, b):
```

```
print(a + b)
```

Función que devuelve:

```
def suma(a, b):  
    return a + b
```

La segunda es mucho más potente porque:

- Permite usar el resultado en otras operaciones.
- Mejora la modularidad.
- Facilita pruebas y reutilización.

3. Llamada e invocación de funciones

◆ ¿Qué significa invocar una función?

Definir una función no la ejecuta.

Para que el código dentro se ejecute, hay que llamarla (invocarla).

👉 Invocar = escribir el nombre + paréntesis.

1 Invocación simple

Se usa cuando la función no tiene parámetros.

```
def saludar():  
    print("Hola")
```

```
saludar()
```

⚠ Error típico:

saludar # ✗ solo devuelve la referencia, no ejecuta la función

Siempre hay que usar paréntesis.

✓ EJERCICIO 1

1) Crear una función que:

- reciba un número
- devuelva su cuadrado

2) Usarla 3 veces con números distintos.

Parte b)

Partiendo de la función `cuadrado(n)` definida en el apartado anterior:

1. Crea una lista llamada `numeros` que contenga al menos cinco números enteros.
2. Recorre la lista utilizando un bucle `for`.
3. En cada iteración, llama a la función `cuadrado()` pasando como argumento el número correspondiente de la lista.
4. Muestra por pantalla el resultado del cuadrado de cada número.

Parte c)

Partiendo de la función `cuadrado(n)` definida anteriormente:

1) Crea una nueva función llamada `cuadrados_lista(lista)` que:

- Reciba como argumento una lista de números.

- Recorra la lista internamente utilizando un bucle `for`.
- Calcule el cuadrado de cada número.
- Devuelva una nueva lista con los cuadrados obtenidos.

2 En el programa principal:

- Crea una lista con al menos cinco números enteros.
- Llama a la función `cuadrados_lista()` pasando la lista como argumento.
- Muestra por pantalla la lista devuelta.

2 Argumentos posicionales

Los valores se asignan por orden.

```
def resta(a, b):  
    return a - b
```

```
print(resta(10, 5)) # 5
```

Aquí:

- 10 → se asigna a `a`
- 5 → se asigna a `b`

⚠ Si cambiamos el orden:

```
print(resta(5, 10)) # -5
```


El resultado cambia.

Idea clave

En argumentos posicionales, el orden importa.

EJERCICIO 2 — Orden básico

Crea una función llamada `multiplicar(a, b)` que:

- 1 Devuelva el resultado de multiplicar `a` por `b`.
- 2 Llámala con:

- `multiplicar(3, 4)`
- `multiplicar(4, 3)`

 Pregunta:

¿El resultado cambia? ¿Por qué?

Parte 2b — Orden que sí cambia el resultado

Crea una función llamada `restar(a, b)` que:

- 1 Devuelva el resultado de `a - b`.
- 2 Llámala con:

- `restar(10, 2)`
- `restar(2, 10)`

 Pregunta:

¿Por qué ahora el orden sí importa?

EJERCICIO 3 — Caso realista (error silencioso)

Crea una función llamada `calcular_salario(sueldo_base, horas_extras)` que:

- Devuelva el salario total sabiendo que cada hora extra vale 15 €.

Después:

- 1 Llama correctamente a la función.
- 2 Llama intercambiando el orden de los argumentos.

 Pregunta:

¿Qué problema puede generar esto en un programa real?

EJERCICIO 4 — Argumentos posicionales con varios parámetros

Crea una función llamada `crear_usuario(nombre, edad, ciudad)` que:

- 1 Devuelva un texto con el siguiente formato: `Ana tiene 25 años y vive en Madrid`
- 2 Llama a la función correctamente pasando los argumentos en el orden adecuado.
- 3 Llama nuevamente a la función intercambiando el orden de los argumentos.
- 4 Observa y explica qué ocurre.

Reflexiones:

1 ¿Cuándo es seguro usar argumentos posicionales?

Es seguro usar argumentos posicionales cuando:

- La función tiene **pocos parámetros** (2 o 3 como máximo).
- El orden es **claro y lógico**.

- Los parámetros no son del mismo tipo o no pueden confundirse fácilmente.
- La función es sencilla y fácil de leer.

👉 Por ejemplo, en operaciones matemáticas simples como:

```
resta(10, 5)
```

Aquí el orden es evidente.

¿Qué riesgo hay si una función tiene muchos parámetros posicionales?

El principal riesgo es:

- Confundir el orden de los argumentos.
- Generar **errores lógicos sin que Python detecte ningún error**.
- Dificultar la lectura del código.
- Hacer el programa más propenso a fallos en proyectos grandes.

👉 Cuantos más parámetros haya, más difícil es recordar su orden exacto.

Esto puede provocar resultados incorrectos sin que el programa muestre ningún error.

Los argumentos posicionales son simples y rápidos, pero cuando la complejidad aumenta, los argumentos nombrados aportan mayor claridad y seguridad.

3 Argumentos nombrados (keyword arguments)

Se indica explícitamente el nombre del parámetro.

```
def presentar(nombre, edad):  
    print(nombre, edad)
```

```
presentar(nombre="Ana", edad=20)
```

Ahora el orden ya no importa:

```
presentar(edad=20, nombre="Ana")
```

- ✓ Más claro
- ✓ Más legible
- ✓ Reduce errores en funciones con muchos parámetros

¿Cuándo sería mejor usar argumentos nombrados?

Es mejor usar argumentos nombrados cuando:

- La función tiene muchos parámetros.
- Varios parámetros son del mismo tipo (por ejemplo, varios números).
- Queremos mayor claridad y legibilidad.
- Trabajamos en equipo y queremos evitar errores silenciosos.
- Queremos evitar depender estrictamente del orden.

👉 Por ejemplo:

```
crear_usuario(nombre="Ana", edad=25, ciudad="Madrid")
```

Aquí queda muy claro qué valor corresponde a cada parámetro.

EJERCICIO 5 — Reescribir el ejercicio anterior

Partimos de:

```
def crear_usuario(nombre, edad, ciudad):  
    return f"{nombre} tiene {edad} años y vive en {ciudad}"
```

✎ Tarea

- ❶ Llama a la función usando argumentos nombrados.
- ❷ Cambia el orden de los argumentos.
- ❸ Comprueba que el resultado sigue siendo correcto.

❹ Valores por defecto

Permiten que un parámetro tenga un valor inicial si no se pasa argumento.

```
def saludar(nombre="Invitado"):
    print("Hola", nombre)
```

```
saludar()           # Hola Invitado
saludar("María")    # Hola María
```

Regla importante

Los parámetros con valor por defecto deben ir después de los obligatorios.

```
def ejemplo(a, b=10):    # ✓ correcto
```

```
def ejemplo(a=10, b):    # ✗ error
```

EJERCICIO 6 — Parámetro opcional

Crea una función llamada `saludar(nombre, idioma)` que:

- Si el idioma es `"es"` → diga `"Hola"`.
- Si el idioma es `"en"` → diga `"Hello"`.
- Si no se pasa idioma → use `"es"` por defecto.

🌟 5 Número variable de argumentos

◆ ***args** → argumentos posicionales variables

Python los agrupa automáticamente en una tupla.

Nota: No confundir, cuando pasamos una lista significa enviar un único argumento estructurado; usar ***args** significa recibir múltiples argumentos individuales que Python agrupa automáticamente en una tupla.

Cuando ponemos un solo asterisco *****, estamos diciendo:

“Esta función puede recibir **cualquier cantidad de argumentos posicionales**.” Recoge todos los argumentos posicionales extra y los guarda en una **tupla**.

```
def sumar(*numeros):  
    return sum(numeros)  
  
print(sumar(2, 4, 6)) # 12
```

Estamos pasando tres valores separados:

- 2
- 4
- 6

Lo que hace Python es: Dentro de la función, los agrupa automáticamente en una tupla.

🟢 EJERCICIO 7 — Sumar cantidad variable de números

✏️ Enunciado

Crea una función llamada `sumar()` que:

- 1 Reciba cualquier cantidad de números.
- 2 Devuelva la suma total.
- 3 Prueba la función con distintas cantidades de argumentos.

EJERCICIO 8 — Promedio variable

Enunciado

Crea una función llamada `promedio()` que:

- ❶ Reciba cualquier cantidad de números.
- ❷ Devuelva el promedio de todos ellos.
- ❸ Si no recibe ningún argumento, devuelva un mensaje indicando que no hay datos.

◆ ****kwargs** → (KeyWord Arguments) argumentos por palabra clave

Si defines:

```
def crear_usuario(**opciones):  
    print(opciones)
```

Le estás diciendo a Python:

“Acepta cualquier cantidad de argumentos con nombre (clave=valor) y guárdalos en un diccionario llamado `opciones`.”

Llamada normal con argumentos nombrados

```
crear_usuario(nombre="Luis", edad=30, activo=True)
```

Escribes argumentos sueltos en formato `clave=valor`.

Python internamente hace esto:

```
opciones = {  
    "nombre": "Luis",  
    "edad": 30,  
    "activo": True  
}
```



Importante:

No estás pasando un diccionario.
Python lo crea automáticamente.

◆ 2 Pasar un diccionario normal (NO es lo mismo)

Si haces esto:

```
crear_usuario({"nombre": "Luis", "edad": 30})
```

Aquí estás pasando:

- Un único argumento
- Que es un diccionario
- Como argumento posicional

Y como la función espera ****opciones**, esto da error ❌

Porque:

****opciones** solo acepta argumentos tipo:

clave=valor

No acepta un diccionario como argumento posicional.

◆ 3 Cómo pasar un diccionario correctamente

Si tienes:

```
datos_usuario = {  
    "nombre": "Luis",  
    "edad": 30,  
    "activo": True  
}
```


Entonces debes hacer:

```
crear_usuario(**datos_usuario)
```

Aquí ****** significa:

“Desempaqueta este diccionario y conviértelo en argumentos clave=valor”.

Python lo transforma en:

```
crear_usuario(nombre="Luis", edad=30, activo=True)
```

Y ahora sí funciona 

◆ Comparación

◆ Caso A — Argumentos nombrados normales

Llamada:

```
crear_usuario(nombre="Luis", edad=30)
```

La función debe definirse con ******

```
def crear_usuario(**opciones):  
    print(opciones)
```

Recibe:

```
{"nombre": "Luis", "edad": 30}
```

 Aquí:

- Tú pasas argumentos nombrados.
 - La función necesita ****** para recogerlos en un diccionario.
-

◆ Caso B — Diccionario sin **

Llamada:

```
crear_usuario({"nombre": "Luis", "edad": 30})
```

✓ La función NO debe llevar **

Debe definirse así:

```
def crear_usuario(datos):  
    print(datos)
```

Recibe:

```
{"nombre": "Luis", "edad": 30}
```

📌 Aquí:

- Estás pasando un único argumento.
- Ese argumento es un diccionario.
- No necesitas ** en la definición.

⚠ Si la defines como:

```
def crear_usuario(**opciones):
```

Y llamas sin **, dará error ✗

◆ Caso C — Diccionario con **

Llamada:

```
crear_usuario(**{"nombre": "Luis", "edad": 30})
```

✓ La función debe definirse con **

```
def crear_usuario(**opciones):  
    print(opciones)
```

Recibe:

```
{"nombre": "Luis", "edad": 30}
```

📌 Aquí:

- Tienes un diccionario.
- Usas `**` al llamar para desempaquetarlo.
- La función necesita `**` para recoger esos argumentos.

Caso	Cómo llamas	Cómo defines la función
A	<code>func(nombre="Luis")</code>	<code>def func(**opciones)</code>
B	<code>func({"nombre": "Luis"})</code>	<code>def func(datos)</code>
C	<code>func(**{"nombre": "Luis"})</code>	<code>def func(**opciones)</code>

- ☐ Si en la llamada ves `clave=valor` → la función necesita `**`.
- ☐ Si estás pasando `{}` directamente → la función NO necesita `**`.
- ☐ Si en la llamada ves `**diccionario` → la función necesita `**`.

Llamada	Qué estás pasando	Qué recibe la función
<code>func(1, 2, 3)</code>	valores separados	tupla (<code>*args</code>)
<code>func(nombre="Luis")</code>	clave=valor	diccionario (<code>**opciones</code>)
<code>func({"nombre": "Luis"})</code>	un diccionario	argumento normal
<code>func(**{"nombre": "Luis"})</code>	diccionario desempaquetado	diccionario (<code>**opciones</code>)

EJERCICIO 9 — Argumentos nombrados normales

Queremos poder hacer esto:

```
registrar_producto(nombre="Portátil", precio=1200, stock=5)
```

Y que imprima:

```
{'nombre': 'Portátil', 'precio': 1200, 'stock': 5}
```

👉 Pregunta:

¿Cómo debe definirse la función `registrar_producto`?

(¿Lleva `**` o no?)

EJERCICIO 10 — Pasando un diccionario normal

Tenemos:

```
datos = {  
  
    "nombre": "Portátil",  
  
    "precio": 1200,  
  
    "stock": 5  
  
}
```

```
registrar_producto(datos)
```

Y queremos que imprima el diccionario.

👉 Pregunta:

¿Cómo debe definirse la función ahora?

(¿Lleva ** o no?)

EJERCICIO 11 — Diccionario desempquetado

Tenemos:

```
datos = {  
  
    "nombre": "Portátil",  
  
    "precio": 1200,  
  
    "stock": 5  
  
}
```

```
registrar_producto(**datos)
```

Y queremos que imprima:

```
{'nombre': 'Portátil', 'precio': 1200, 'stock': 5}
```

👉 Pregunta:

¿Cómo debe definirse la función en este caso?

(¿Lleva ** o no?)

♦ Orden obligatorio

En la definición:

```
def funcion(a, *args, **kwargs):
```

Siempre:

1. parámetros normales
2. *args
3. **kwargs

Resumen:

Tipo	¿Importa el orden?	¿Más legible ?	Ejemplo
Posicional	Sí	Medio	<pre>def resta(a, b): return a - b resta(10, 5)</pre>
Nombrado (keyword)	No	Alto	<pre>def presentar(nombre, edad): print(nombre, edad) presentar(edad=20, nombre="Ana")</pre>
Por defecto	Flexible.	Alto	<pre>def saludar(nombre="Invitado"): print("Hola", nombre) saludar() # Hola Invitado saludar("María") #Hola María</pre>
*args	Cantidad variable. Se reciben como una tupla .	Medio	<pre>def sumar(*numeros): return sum(numeros) sumar(2, 4, 6)</pre>
kwargs (KeyWord Arguments)	Clave–valor variable. Se reciben como un diccionario .	Alto	<pre>def mostrar_info(datos): for k, v in datos.items(): print(k, v) mostrar_info(nombre="Ana", edad=25)</pre>



4. La instrucción **return**

◆ ¿Qué hace **return**?

La instrucción **return** cumple dos funciones fundamentales:

- 1 Devuelve un valor al lugar donde se llamó la función.
- 2 Finaliza inmediatamente la ejecución de la función.



Ejemplo básico

```
def suma(a, b):  
    return a + b
```

```
resultado = suma(3, 5)  
print(resultado)
```

Aquí ocurre esto:

- Se llama a **suma(3, 5)**
- La función calcula **3 + 5**
- **return** devuelve **8**
- La ejecución vuelve al punto donde fue llamada



¿Qué pasa si NO usamos **return**?

```
def suma(a, b):  
    a + b
```

```
print(suma(3, 5))
```

Salida:

None

¿Por qué?

Porque en Python:

Si una función no tiene `return`, devuelve automáticamente `None`. Y esto genera muchos errores lógicos.

Diferencia CLAVE: `print` vs `return`

Función que imprime:

```
def suma(a, b):  
    print(a + b)  
  
resultado = suma(3, 5)  
print(resultado)
```

Salida:

8

None

¿Por qué?

Porque:

`print` muestra el valor

pero no lo devuelve

la función sigue devolviendo `None`

Función que devuelve:

```
def suma(a, b):  
    return a + b
```

Ahora sí podemos hacer:

```
total = suma(3, 5)  
print(total * 2)
```

Y funciona correctamente.

return y flujo de ejecución

Cuando Python encuentra `return`, la función termina inmediatamente.

```
def ejemplo():  
    print("Inicio")  
    return  
    print("Nunca se ejecuta")
```

La segunda impresión nunca ocurre.

👉 `return` corta el flujo.

Ejemplo con condición

```
def verificar(numero):  
    if numero < 0:
```

```
        return "Negativo"

    return "Positivo"
```

Aquí:

- Si se cumple la condición → termina ahí.
- Si no → sigue hasta el siguiente `return`.



Retorno múltiple

Python permite devolver varios valores usando tuplas.

```
def operaciones(a, b):
    return a + b, a - b

suma, resta = operaciones(10, 5)
print(suma)    # 15
print(resta)   # 5
```

Internamente se devuelve:

```
(15, 5)
```



Funciones sin retorno

```
def saludar(nombre):
    print("Hola", nombre)
```

Son útiles cuando:

- Solo realizan una acción.

- No necesitamos usar el resultado después.

Pero abusar de ellas reduce modularidad.



Resumen para apuntes

- `return` devuelve un valor.
- Sin `return`, la función devuelve `None`.
- `return` finaliza la ejecución.
- `return` Permite reutilización real del resultado.
- `return` Mejora modularidad y depuración.



Ejercicio:

¿Qué imprime este código?

```
def doble(n):  
    print(n * 2)  
  
resultado = doble(4)  
print(resultado)
```

Respuesta esperada:

8
None



Mini reto práctico

Reescribe esta función para que sea reutilizable:

```
def calcular_area(base, altura):  
    print(base * altura)
```

Objetivo:

Que podamos hacer:

```
area = calcular_area(5, 3)  
print(area + 10)
```